

Exercise 1: Setting Up JUnit

```
<dependencies>
```

```
<dependency>
```

```
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>4.13.2</version>
<scope>test</scope>
```

```
</dependency>
```

```
</dependencies>
```

Output

The screenshot shows the IntelliJ IDEA IDE interface. The top toolbar includes icons for File, Edit, Run, and other IDE functions. The main editor area displays the `pom.xml` file for the `JUnitTest` project. The XML content is as follows:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>org.example</groupId>
  <artifactId>JUnitTest</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

The Run output console at the bottom shows the following log messages:

```
[INFO] Archetype repository not defined. Using the one from [org.apache.maven.archetypes:maven-archetype-quickstart:1.1]
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.1
[INFO] Parameter: basedir, Value: C:\Users\monda\AppData\Local\Temp\archetype670f9af6b06794a5e65414f04b4dd56ftmp
[INFO] Parameter: package, Value: org.example.test
[INFO] Parameter: groupId, Value: org.example.test
[INFO] Parameter: artifactId, Value: JUnitTest
[INFO] Parameter: packageName, Value: org.example.test
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: C:\Users\monda\AppData\Local\Temp\archetype670f9af6b06794a5e65414f04b4dd56ftmp
[INFO] BUILD SUCCESS
[INFO] Total time: 3.609 s
[INFO] Finished at: 2026-06-22T23:02:43+05:30
```

Exercise 3: Assertions in JUnit

Code--

```
package org.example.test;

import org.junit.Test;
import static org.junit.Assert.*;

public class AssertionsTest {

    public static class assertiontest {
        @Test
        public void testAssertions() {
            // Assert Equals
            assertEquals(10, 5 + 5);

            // Assert Not Equals
            assertNotEquals(10, 5 + 4);

            // Assert True
            assertTrue(20 > 10);

            // Assert False
            assertFalse(20 < 10);

            // Assert Null
            String str1 = null;
            assertNull(str1);

            // Assert Not Null
            String str2 = "JUnit";
            assertNotNull(str2);

            // Assert Same
            String a = str2;
            assertSame(str2, a);

            // Assert Not Same
            String b = new String("JUnit");
            assertNotSame(str2, b);

            // Assert Array Equals
            int[] expected = {1, 2, 3};
            int[] actual = {1, 2, 3};
            assertEquals(expected, actual);
        }
    }
}
```

Output Exercise 3

The screenshot displays an IDE interface with the following components:

- Project Explorer:** Shows a project named 'JUnitTest' with a package structure including 'org.example.test' and 'org.example.test.test'. The 'test' package is expanded, showing 'AppTest', 'AssertionsTest', and 'CalculatorServiceTest'.
- Code Editor:** Displays the 'AssertionsTest.java' file. The code defines a class 'AssertionsTest' with a static class 'assertiontest' containing a 'testAssertions()' method. The method performs several assertions:

```
public class AssertionsTest {  
    public static class assertiontest {  
        public void testAssertions() {  
            assertEquals( unexpected: 10, actual: 5 + 4);  
  
            // Assert True  
            assertTrue( condition: 20 > 10);  
  
            // Assert False  
            assertFalse( condition: 20 < 10);  
  
            // Assert Null  
            String str1 = null;  
            assertNull(str1);  
  
            // Assert Not Null  
            String str2 = "JUnit";  
            assertNotNull(str2);  
  
            // Assert Same  
            String a = str2;  
            assertEquals(str2, a);  
  
            // Assert Not Same  
            String b = new String( original: "JUnit");  
            assertNotSame(str2, b);  
        }  
    }  
}
```
- Run Console:** Shows the execution of 'AssertionsTest\$assertiontest.testAssertions'. The output indicates '1 test passed' and '1 test total, 7 ms'. The process finished with exit code 0.
- Bottom Bar:** Displays the file path 'JUnitTest > src > test > java > org > example > test > AssertionsTest' and the encoding 'UTF-8'.

Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

Code--

```
package org.example.test;
import org.junit.*;

public class CalculatorServiceTest {
    @BeforeClass
    public static void init(){
        System.out.println("This is the Beginning of Testing .....");
    }
    @Before
    public void Display(){
        System.out.println("Testing....");
    }
    @Test
    public void add(){
        int actualRes = CalculatorService.add(5,5);
        int expectedRes = 10;
        Assert.assertEquals(expectedRes, actualRes);
    }
    @Test
    public void product(){
        int result = CalculatorService.product(5,5);
        Assert.assertEquals(25, result);
    }

    @Test
    public void subtract(){
        int result = CalculatorService.subtract(10,5);
        Assert.assertEquals(5, result);
    }
    @After
    public void display(){
        System.out.println("TestCase passed");
    }
    @AfterClass
    public static void afterClass() {
        System.out.println("All Testcases are passed , You are good to go :)");
    }
}
```

Output 4

